

Manual VLA: Methodology and AI Architecture

A Lightweight Vision-Language-Action Transformer for Robotic Manipulation

Autonomous Robotics & VLA Research Lab

https://github.com/krazykarthik2/manual_vla

September 2026

1. Abstract

This paper describes the Manual VLA system, a lightweight Vision-Language-Action transformer designed for real-time robotic manipulation on a 4-DOF Dobot Magician. The architecture uses a custom patch-based vision encoder, Hadamard MLP intent conditioning, and a compact transformer encoder to directly predict end-effector delta actions from a single overhead RGB image and a structured intent vector.

2. System Overview

Manual VLA Architecture

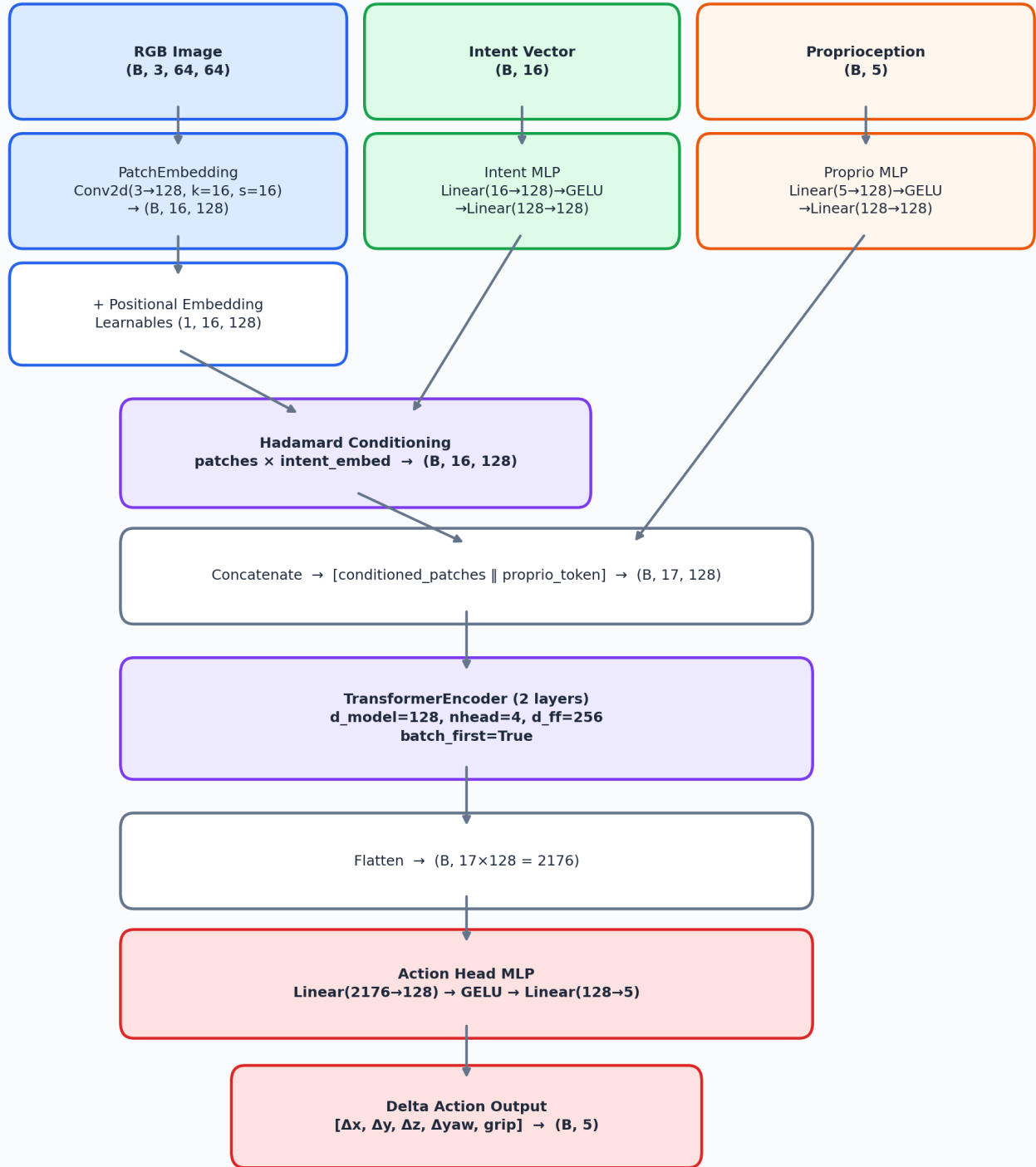


Figure 1: Manual VLA Architecture - Patch Embedding, Hadamard Conditioning, Transformer Encoder.

Manual VLA is the foundational architecture in the repository. Unlike SmolVLA or Full SmolVLA, it does not use any pretrained vision-language backbone. Instead, it employs a fully trainable, from-scratch vision encoder combined with a structured intent vector to condition actions.

The system pipeline is:

1. Capture a 64x64 RGB overhead image of the workspace.
2. Encode the image into patch tokens using a convolutional patch embedding.
3. Encode a 16-dimensional intent vector specifying action type, source color, and target color.
4. Apply Hadamard (element-wise) conditioning of visual tokens with intent features.
5. Concatenate a proprioceptive state token and process through a transformer encoder.
6. Predict a 5-dimensional delta action: [dx, dy, dz, dyaw, gripper].

3. Architecture Details

3.1 Patch Embedding (PatchEmbedding)

Input images of shape (B, 3, 64, 64) are processed by a Conv2d layer with kernel_size=16 and stride=16, producing (B, embed_dim, 4, 4) feature maps. These are flattened into 16 patch tokens of dimension embed_dim=128.

```
Conv2d(3, 128, kernel_size=16, stride=16)
-> flatten(2).transpose(1, 2)
-> Output: (B, 16, 128)
```

3.2 Intent Encoding

The intent vector is 16-dimensional, composed of:

- Action type (2D one-hot): pick_place or push
- Source object color (7D one-hot): red, blue, yellow, green, purple, orange, cyan
- Target destination color (7D one-hot): same 7 colors

This is projected through a 2-layer MLP with GELU activation:

```
Linear(16 -> 128) -> GELU -> Linear(128 -> 128)
```

3.3 Proprioception Encoding

The 5-dimensional proprioceptive state (current end-effector position + gripper state) is projected through the same MLP structure:

```
Linear(5 -> 128) -> GELU -> Linear(128 -> 128)
```

This produces a single proprioceptive token of dimension 128.

3.4 Hadamard MLP Conditioning

Instead of cross-attention, Manual VLA uses element-wise (Hadamard) multiplication to condition the visual patch tokens with the intent embedding:

```
conditioned_patches = patches * intent_embed(intent).unsqueeze(1)
```

This modulates each patch token's activation pattern based on the task intent, encouraging the vision encoder to focus on task-relevant spatial regions.

3.5 Transformer Encoder

The conditioned visual patches (16 tokens) and the proprioceptive token (1 token) are concatenated into a sequence of 17 tokens, each of dimension 128.

This sequence is processed by a 2-layer TransformerEncoder:

- d_model = 128
- nhead = 4
- dim_feedforward = 256
- batch_first = True

3.6 Action Head

The transformer output is flattened across all 17 tokens ($17 * 128 = 2176$ dimensions) and passed through a 2-layer MLP:

Linear(2176 -> 128) -> GELU -> Linear(128 -> 5)

Output: 5D delta action [dx, dy, dz, dyaw, gripper].

4. Training Methodology

Manual VLA is trained with Optimal Transport Conditional Flow Matching (OT-CFM). Demonstrations are auto-generated by a rule-based oracle that produces pick-and-place trajectories with random object placements and visual distractors.

The training pipeline:

1. Generate demonstrations using auto_generate_demos.py
2. Train the model with train_flow.py using AdamW optimizer and cosine annealing
3. Evaluate with run.py in the simulated Dobot environment

5. Flow Matching Formulation

Given a clean demonstration trajectory x_1 and noise $x_0 \sim N(0, I)$, the interpolated sample at time t is:

$$x_t = (1 - t) * x_0 + t * x_1, \quad t \sim U(0, 1)$$

The target velocity field is:

$$u_t = x_1 - x_0$$

The model learns to predict this velocity field conditioned on the current image and intent. At inference, trajectories are generated by Euler integration from pure noise over K steps.